

MATHEMATICAL MODELING (CO2011), SEMESTER 261 (2026–2027)

Semester-long Integrated Assignment

Invigilator Assignment Problem

One real problem, four modules:

Logic $\Rightarrow$ Linear & Integer Programming $\Rightarrow$ Automata $\Rightarrow$ Dynamical Systems

Instructors (CO2011 teaching team):

Nguyen An Khuong Mai Xuan Toan Tran Hong Tai Pham An Vinh
Le Hong Trang Vu Trung Hieu Vu Ngoc Tu

Quick Start: read this first

What. One semester-long team project: build a small tool that assigns exam *invigilators* on real Faculty data, using all four CO2011 modules (Logic $\rightarrow$ LP/ILP $\rightarrow$ Automata $\rightarrow$ Dynamics) wrapped in a Streamlit web app.

Your five milestones (weeks run Mon–Sun; push the tag by the Sunday):

Tag	Due	Deliver
m1	20 Sep	Logic: predicate spec + SAT of the rules
m2	11 Oct	LP/ILP model + solver that beats the baseline (<i>before the midterm</i>)
m3	8 Nov	Automata: DFAs for temporal rules + pumping-lemma proof
m4	29 Nov	Dynamics: recurrence, equilibrium & stability
m5/submission	13 Dec	Web app + report + video

Every week (Process & Ownership = 18%). Push a tagged commit `week-01...week-14` and add one dated line to `CHECKPOINTS.md`. Configure git as `user.name="<StudentID> <Name>"`, `user.email=<id>@hcmut.edu.vn`.

Team ID (self-formed, unique). CO2011-261-`<group>`-`<smallest student ID>`, `group` $\in$ {L,CC,A01,TN01} (L and CC classes may team across sections), e.g. CO2011-261-L-2252107. **Seed:** `python tools/make_seed.py <TEAM_ID> > data/seed.txt` (no seed is handed out).

Why it matters. On paper 20%, but the midterm and final are drawn from these same four modules, so doing the project well is also how you pass the exams.

What to read when. *This week:* §1 (context + data), §2 Module 1, §3–§4 (timeline, repo & ownership rules). *Before each module:* its part of §2 and the self-study row in §1.4. *As reference:* §6 rubric (self-grade with `rubric.yaml`), §8 glossary, §9 references, Appendix A/B.

Dear students, this semester's assignment is a **single, semester-long team project** (teams of **4–5**) that weaves *all four theory modules of CO2011* into one real-world problem, the **Invigilator Assignment Problem (IAP)**, using actual anonymized data from the Faculty. You will **specify** the rules in logic, **solve** one exam period with Integer Linear Programming, handle **temporal** rules with automata, run the schedule **across the whole term** as a discrete dynamical system, and **integrate** everything into a small web app. It is a project worthy of a top engineering school: done well, it should make you proud.

Read this before you plan

This project is **light if you work like a professional team, and nearly impossible if you cram**. Fourteen working weeks, five milestones, weekly commits. A team that plans in short *agile* iterations, meaning plan then build then review *every week*, that splits ownership early and integrates continuously, will find each module very manageable. A team that starts in December will **not** finish. Since the assignment is **20% of the course grade** and its content is examined in the **final**, that team is very likely to **fail the course**. Treat every weekly checkpoint as a real sprint boundary.

A first for CO2011: your approach must be different this year

This is the **first semester** in which Mathematical Modeling runs as **one project spanning the whole term**, released in **Week 1** instead of appearing as a short two-week task after the midterm. Habits and files from previous years do not carry over: there is no end-of-term version to rush, no shared instance to reuse, and the grade now rewards *steady weekly work and demonstrable individual ownership* rather than a last-minute report. Start on Day 1, commit every week, and build the four modules as one growing codebase. A team that treats this like an old-style assignment will fall behind and cannot catch up.

1. Problem, Context, and How to Prepare

Each semester the Faculty runs *hundreds* of exam sessions across two campuses and must assign a required number of *invigilators*¹ to each, honoring hard regulations (capacity, no double-booking, availability) and softer wishes (location preference, fatigue limits, fairness). Done by hand it takes days and still yields lopsided workloads. The IAP is a Constraint Satisfaction Problem that has been studied in the operations-research literature [1, 2, 3], and it is **NP-hard**²; your team will build the mathematical machinery that solves it in seconds, and, more importantly, understand *why* it works.

The smallest instance (to fix ideas)

- **Invigilators** $I = \{\text{CB1, CB2, CB3}\}$; **Shifts** $J = \{\text{Morning, Afternoon}\}$, each needing 2 invigilators.
- **Availability**: CB1 is busy in the morning ($x_{11} = 0$). **Variables**: $x_{ij} \in \{0, 1\}$ (1 iff i works shift j).
- **Objective**: balance the workload (minimize its spread) among staff.

The real dataset is three orders of magnitude larger; the same model must scale to it.

Dataset, baseline, and what you must simulate

Use the anonymized dataset: [Anonymized Dataset](#). It contains an existing schedule that is your **baseline**: extract parameters, then improve upon it. It directly supplies Modules 1–2 (sets, availability, capacities, the assignment to beat).

¹Invigilator (exam proctor): the staff member assigned to supervise each exam session; this is the object being scheduled. Problem background: [1].

²Constraint Satisfaction Problem (CSP): find an assignment satisfying all constraints. NP-hard means at least as hard as every problem in NP, with no known polynomial-time algorithm. See [15], Part I.

Modules 3–4 should be driven by the *same* real data wherever possible, not invented from scratch. Each row of the baseline records *who* was assigned to *which* session, and each session carries a date/time and a campus, so:

- **Module 3 (real temporal strings).** Sort each invigilator’s assigned sessions by date and time to *reconstruct their actual sequence* over $\Sigma = \{M, A, N, -\}$. Test your DFA rules on these real strings; you only *choose* the rules, the strings come from the data. If the file covers a short window, tile it into a multi-week horizon and calibrate the tiling to the observed per-day frequencies.
- **Module 4 (fit, don’t fabricate).** Estimate the dynamical parameters from the data: workload w_n is the running total of real assignments; availability transition proportions (available/busy/on-leave) are estimated from how often each person appears or drops out across sessions; the fatigue coefficients (a, b, r) are calibrated so the equilibrium matches the observed average load. The model is then *fit* to reality rather than guessed.

Only the pieces the dataset genuinely lacks may be *simulated*, and every such assumption must be stated: (i) location preferences in three categories (near Campus 1 prefers CS1, near Campus 2 prefers CS2, balanced has no preference); (ii) any night shifts not present in the file; (iii) soft-constraint weights (the three values printed by `python tools/make_seed.py <TEAM_ID> --weights`, in $[0.5, 2.0]$). **Each team’s seed, and the soft-constraint weights derived from it, is computed from its own team ID**, so that no two teams share the same instance (see §4).

Why a web app in a mathematics course?

Modeling is only useful when its result reaches a decision-maker, and a scheduler will never open your .lp file or read your proof. They open a page, pick a week, and see the plan, the fairness curve, and a red flag if a rule is broken. The web layer is exactly where your four modules become *one usable tool*: logic (is it valid?), ILP (the plan), automata (is it temporally legal?), and dynamics (is it fair over the term?). Technically it is **light**: Streamlit turns a Python function into a web page in a few lines, with no HTML, no JavaScript, and no servers, so a second-year student spends effort on the *model*, not the plumbing. This mirrors real operations-research practice, where the solver is wrapped in a decision-support dashboard.

Self-study roadmap (read before you need it)

Each module’s assignment work runs *in parallel* with its lectures, and you need the key ideas from the *start* of each window while the lectures are still building up, so reading a little ahead is essential. Chapters below are from the official CO2011 teaching plan; past exams are in the [course drive](#).

Module	When	Read first (self-study)
M1 Logic	Sep 7–20	Huth & Ryan [8], Ch. 1, 2, 4; Rosen [7], Ch. 1 (logic & proofs) and §5.5 (program correctness / verification); further Ben-Ari [9], Ch. 15.
M2 LP/ILP	Sep 21–Oct 11	Matoušek & Gärtner [14], Ch. 1–5; further Nemhauser & Wolsey [15] (integer). <i>Read as M2 opens; Optimization is lectured these same weeks, just before the midterm.</i>
M3 Automata	Oct 19–Nov 8	Linz [11], Ch. 1–4; Rosen [7], Ch. 13 (modeling computation: grammars, finite-state machines); further Hopcroft, Motwani & Ullman [12]; Kandar [13] (MCQ practice).
M4 Dynamics	Nov 9–29	Giordano, Fox & Horton [16], Ch. 1 (difference equations); further Elaydi [17].

Rosen [7] covers Modules 1 and 3 in one book and ships with a *Student’s Solutions Manual*, so it is the most efficient single source for self-checking your logic, verification, and automata practice.

Free online tools & extra MCQ practice

LP/ILP: [SolverWorld online solver](#). **Automata:** [free DFA/NFA/regex tools](#); MCQ practice in Kandar [13], in Rosen [7] Ch. 13, and a [Formal-Languages-and-Automata MCQ set](#). **Dynamics:** a [dynamical-systems mind-map](#) and [logistic-model exercises](#) (Calculus §8.6). **General:** the [course drive](#) (references, textbooks, past exams). Use tools to *check* your work, never to replace your own modeling.

2. The Modules: four theory modules plus integration

The single thread is **Specify (Logic), Solve one period (ILP), Handle time (Automata), Run across the term (Dynamics), Integrate (Web app)**. Every module *extends the same model and the same repository*; do not start a new problem each time.

Module 1: Logic (specification)

- 1.1 Predicate specification.** Introduce predicates ($\text{Assign}(i, j)$, $\text{Busy}(i, j)$, $\text{Overlap}(j, k)$, $\text{AtCampus}(j, c)$, $\text{Prefer}(i, c)$) and write every *hard* rule as a first-order formula³, for example no double-booking of overlapping shifts (with Overlap symmetric and irreflexive, or the guard $j \neq k$): $\forall i \forall j \forall k (\text{Overlap}(j, k) \wedge \text{Assign}(i, j) \rightarrow \neg \text{Assign}(i, k))$.
- 1.2 Propositional encoding & SAT.** For the toy instance and a small slice of the data, encode feasibility in CNF⁴ and decide satisfiability with a SAT/SMT solver (python-sat or z3); report a model or a minimal unsatisfiable core.
- 1.3 Logic-to-LP bridge.** Translate each clause into a linear (in)equality over 0/1 variables. A disjunctive clause maps to one “ ≥ 1 ” inequality; *counting* rules (exact capacity = 2, at-most- k) are cardinality / pseudo-Boolean constraints, which are one inequality in LP but many clauses in pure CNF. This table feeds Module 2.

Worked example: Module 1

Two invigilators $\{\text{CB1}, \text{CB2}\}$, one shift s needing exactly one person; CB1 is busy at s . Rules: $\text{Busy}(\text{CB1}, s) \rightarrow \neg \text{Assign}(\text{CB1}, s)$ and “exactly one assigned”. With $a_i = \text{Assign}(\text{CB}i, s)$, CNF is $(\neg a_1) \wedge (a_1 \vee a_2) \wedge \neg(a_1 \wedge a_2)$; a SAT solver returns $a_1=0, a_2=1$. The bridge: clause $(a_1 \vee a_2) \Rightarrow a_1 + a_2 \geq 1$; capacity $\Rightarrow a_1 + a_2 = 1$.

Module 2: Linear & Integer Programming (the core)

- 2.1 Parameters:** extract all sets and parameters (including simulated location data).
- 2.2 Decision variables:** state types (binary/integer/continuous) and meaning.
- 2.3 Objective:** maximize fairness or minimize travel penalties (or your own, justified). *Workload variance is quadratic*, so to keep an *ILP* use a linear fairness surrogate: min-max load ($\min t, t \geq w_i \forall i$) or the L_1 deviation ($\min \sum_i d_i, d_i \geq w_i - \bar{w}, d_i \geq \bar{w} - w_i$). Minimizing true variance makes it an MIQP⁵, for which you would use SCIP.
- 2.4 Hard constraints⁶:** capacity, no double-booking, availability (from Module 1).

³Predicate and first-order formula: a logical expression with variables and quantifiers $\forall, \exists$ that evaluates to true or false. See [8], Ch. 2; [7], §1.4–1.6.

⁴CNF (conjunctive normal form): a conjunction of disjunctive clauses. SAT: decide whether a Boolean formula is satisfiable. Unsat core: a minimal subset of clauses that is still unsatisfiable. See [8], §1.4–1.5; [10].

⁵The fairness term is linearized via a min-max or L_1 objective to keep the model an ILP; minimizing the true (quadratic) variance would make it an MIQP (mixed-integer quadratic program). See [14], Ch. 1–3.

⁶Hard constraint: must be satisfied. Soft constraint: may be violated but is penalized in the objective. See [4].

- 2.5 Soft constraints:** location preference, fatigue limits, as penalties.
- 2.6 Data preprocessing:** a Python script reading the dataset and generating synthetic preferences.
- 2.7 Solver:** implement in a free solver (PuLP, OR-Tools, or SCIP).
- 2.8 Tuning:** solve; if infeasible, relax or re-weight.
- 2.9 Performance:** complexity, runtime, comparison against the baseline.
- 2.10 Counter-example (a wrong model).** Exhibit *one* plausible but wrong modeling choice (for instance dropping the overlap constraint, or relaxing to a continuous LP) and give a concrete counter-example schedule it admits, showing why it is invalid.

Worked example: Module 2

Three staff members, four shifts each needing one invigilator, each staff member at most two shifts, minimize the maximum load. With $x_{ij} \in \{0, 1\}$, $\sum_i x_{ij} = 1 \forall j$, $w_i = \sum_j x_{ij} \leq 2$, objective $\min t$ s.t. $t \geq w_i$. Optimum: loads $(2, 1, 1)$, $t = 2$, which is perfectly balanced, and every constraint stayed *linear*.

Milestone: Modules 1–2 complete by Oct 11 (before the midterm week).

Module 3: Automata & Formal Languages (temporal rules)

- 3.1 Schedule as a string.** Encode each invigilator’s term as a string over $\Sigma = \{M, A, N, -\}$ (morning/afternoon/night/off, *one symbol per day*, so $\#_M, \#_A$ count days with a morning/afternoon shift) and express ≥ 3 sequential rules as regular languages, building a DFA⁷ for each: for example “no > 2 consecutive working days”; “a night shift is followed by a rest day”; “at most k shifts in any 7-day window”. *Submit each DFA* as a transition-table JSON in `m3_automata/` (states, alphabet, start, accepting, transitions); the hidden grading test set is generated and labeled directly from your declared automata, so the format is fixed even though the rules and k are yours.
- 3.2 Product & minimization.** Intersect the DFAs and minimize the product; report state counts (Myhill-Nerode).
- 3.3 Regular constraint $\rightarrow$ ILP.** Unfold the minimized DFA into a layered graph and derive the flow/ILP constraints enforcing acceptance (the *regular* global constraint [5]; explicit MIP form [6]). You may instead use OR-Tools’ `AddRegular` / automaton constraint and *explain* the decomposition. Add these rules to your Module 2 model and re-solve; compare.
- 3.4 Regular vs. non-regular fairness (the finite-memory boundary).** Consider morning/afternoon balance over the term. (a) Show the *realistic* rule “the morning and afternoon counts never differ by more than c ”, i.e. $|\#_M - \#_A| \leq c$, **is regular**, and build its DFA (a bounded counter with states $-c, \dots, c$ plus a trap). (b) Show the *idealized* exact-balance rule $L = \{w : \#_M(w) = \#_A(w)\}$ is **not** regular: since $L \cap M^*A^* = \{M^nA^n\}$, apply the pumping lemma⁸ to the latter and use closure under intersection. (c) Conclude why real rosters enforce the bounded, finite-state version, while an *exact* global count needs unbounded memory and so must be pushed into the ILP, not a finite automaton.

Worked example: Module 3

Rule “no > 2 consecutive working days” over $\Sigma = \{W, -\}$. A DFA needs four states: q_0, q_1, q_2 (current run of 0, 1, 2 working days) and a dead state; on W , $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow \text{dead}$; on $-$, every live state $\rightarrow q_0$. Accepting = $\{q_0, q_1, q_2\}$. It accepts WW - WW , rejects WWW . To feed the ILP, lay one node per (day, state); a path from start to an accepting state is a legal schedule, and flow-conservation

⁷Regular language: the class of languages recognized by a deterministic finite automaton (DFA). See [11], Ch. 2–3; [7], Ch. 13.

⁸Pumping lemma: a tool for proving a language is *not* regular (if a substring can be “pumped” while the string must stay in the language, a contradiction follows). See [11], §4.3; [12], §4.1.

constraints force the string to be accepted.

Module 4: Discrete Dynamical Systems (across the term)

Choose exactly ONE direction, justify it, and for it: derive the recurrence⁹ from the data, iterate it numerically, find fixed points $p^* = f(p^*)$, classify stability, and connect the long-run behavior to fairness, verifying every claim with code. *The dynamics must be genuine*: exhibit a map with a real fixed point whose stability is not automatic, not a driven accumulator.

- **A: Closed-loop rolling-horizon fatigue (recommended).** Re-solve week by week, but *close the loop*: let each week's ILP penalize assigning load to already-tired staff, so the fatigue-penalty weight $\lambda_i(F_{i,n})$ grows with the current fatigue $F_{i,n}$. Then the week- n solution is $x_n = g(F_n)$ and $F_{n+1} = aF_n + b \cdot \text{load}(g(F_n)) - r \cdot \text{rest}$, $0 < a < 1$, is a genuine closed-loop map. Find its fixed point, classify stability (now *not* automatic), and study the response to a perturbation such as a sudden sick leave. *Note*: the raw cumulative workload $w_{n+1} = w_n + A_n x_n$ has eigenvalue 1 and *no* non-trivial fixed point, so the real dynamics lives in the fatigue feedback, not the accumulator.
- **B: Vector fatigue & fairness (lighter).** Take the fatigue recurrence for the *whole team as a vector* $F_n \in \mathbb{R}^{|I|}$ and analyze the *spread* (max–min or variance) as the quantity that converges. Show that equal weekly load drives every component to the *same* equilibrium $F^* = \frac{b\bar{\ell} - r\bar{\rho}}{1 - a}$, so the spread $\rightarrow 0$: fairness *is* the stable fixed point. Add “keep every trajectory below a threshold” as a soft constraint.
- **C: Availability Markov chain & mixing.** Model each invigilator's availability (available/busy/on-leave) as a Markov chain; *verify* irreducibility and aperiodicity *from the data* and compute the steady state. Because convergence is then guaranteed, the real question is the *rate*: estimate the mixing time from the second-largest eigenvalue modulus (how many weeks until availability is within tolerance of steady state), and feed the steady state as expected-availability parameters into the ILP.

Worked example: Module 4

Fatigue $F_{n+1} = 0.8 F_n + 2$ (one shift/week, $r=0$), $F_0=0$, gives $0, 2, 3.6, 4.88, \dots \rightarrow F^* = \frac{2}{1-0.8} = 10$; since $|0.8| < 1$ the equilibrium is stable. Fairness link: staff with equal weekly load converge to the *same* F^* , whereas unequal loads give different equilibria, which is exactly the unfairness the ILP objective fights.

Module 5: Integration & Web Application

Build a **simple Streamlit app** (a single `app.py` is fine) that ties the four modules: select/upload the dataset; run the SAT feasibility check (M1); solve the ILP (M2); enforce and display the automaton-based temporal validity (M3); simulate and plot the dynamical trajectory (M4). Show at least: the weekly assignment table, a fairness/variance-over-time chart, the fatigue/equilibrium (or availability) trajectory, and a “schedule valid (DFA-accepted)” flag. Keep it light: set solver time limits, cache results (`st.cache_data`), run the in-app solve on a toy/subset instance, and precompute heavy runs offline. Start the `app.py` skeleton early (around Week 6) and grow it as each module finishes.

⁹Difference equation / recurrence $x_{n+1} = f(x_n)$; an equilibrium (fixed point) is $p^* = f(p^*)$; it is stable if nearby states converge to it. See [16], Ch. 1 (Modeling Change); [17], Ch. 1–2.

3. Timeline & Weekly Checkpoints

All weeks run Monday–Sunday; dates follow the official 2026–2027 academic calendar (semester teaching 24 Aug–13 Dec; National Day 2 Sep off; centralized midterm week 12–18 Oct with no classes; final exams 14–28 Dec).

Phase	Dates (Mon–Sun)	Module	GitHub checkpoint
Preparation	24 Aug–6 Sep	Read brief, form team, self-study	Create repo, README, team table
M1	7–20 Sep	Logic (specification)	tag m1; weekly tags week-01...02
M2	21 Sep–11 Oct	Linear/Integer Programming	tag m2 (before midterm)
<i>Midterm</i>	12–18 Oct	<i>Logic, Verification, Optimization</i>	
M3	19 Oct–8 Nov	Automata & Formal Languages	tag m3
M4	9–29 Nov	Discrete Dynamical Systems	tag m4
M5	30 Nov–13 Dec	Integration + web app + report	tag m5, then submission
<i>Final</i>	14–28 Dec	<i>Automata, Dynamics + all 4 assignment modules</i>	

Weekly rule: by the end of *every* week each team pushes a tagged commit (week-01...week-14) and appends a dated entry to CHECKPOINTS.md (progress plus each member’s tasks). Milestone tags m1–m5 mark module completion.

Definition of Done (every milestone)

A milestone counts as *done* only when, in its tagged commit: (i) the code runs from a clean clone via `pip install -r requirements.txt` and *one* documented command; (ii) the numbers in the report match what the code produces (no hand-edited results); (iii) every claim, whether an optimum, a DFA acceptance, or an equilibrium, is backed by a runnable check; (iv) the CHECKPOINTS.md and CONTRIBUTIONS.md entries for that module are complete; (v) each contributing member has at least one authored commit on that module.

4. Repository, Commits & Ownership

Submission is a GitHub repository, not a ZIP. Create your team repo from the **course template repository** (link on the course page): click “Use this template” → *Create a new repository* (a fresh, independent private repo, **not a fork**), name it your team ID, keep it **private**, and add **your class’s instructor** as a collaborator by 6 Sep. Submit the repository URL on the LMS; the commit tagged **submission** is your final submission. **Do not fork, and do not start from an empty repo:** the template already ships `run_all.py`, `tools/make_seed.py`, the folder layout, and this rubric, so the autograder finds everything where it expects and your own history stays clean for ownership grading.

! CRITICAL: team ID, git identity & seed drive automatic grading

An incorrect team ID or git identity means the autograder cannot match your team or your commits, so your work may go ungraded and force manual rework for the instructors, which is penalized. Follow this section *exactly*.

1. Team formation & ID (form it yourselves on Day 1, no sign-up sheet). Teams of 4–5 may be formed *across sections within the same group*: the **L group** (L01, L02, L03) may team up together, and the **CC group** (CC01–CC05) may team up together; A01 and TN01 each form teams within their own section. Your team ID then follows one fixed rule that makes collisions impossible: take your **group code** (L, CC, A01, or TN01) and append the **smallest student ID among your members**. Format C02011-261-{group}-{minStudentID}, for example C02011-261-L-2252107 or C02011-261-CC-2253010. Because each student is in exactly one team, that smallest ID is unique, so your team ID is automatically unique with no central coordination. Use this exact code everywhere (byte-for-byte: case, hyphens, group token), because the seed is a hash of it: repo name, report cover, slides, video title, LMS submission. **Freeze it at registration (6 Sep)**; later changes in membership do *not* change the team ID or seed.

2. Per-member git identity. Before your first commit, *each* member configures git so every commit is traceable to a person and a school account:

```
git config user.name "2252345 Nguyen Van A"
git config user.email "2252345@hcmut.edu.vn"
```

That is your **student ID + full name** as the name and your **@hcmut.edu.vn email** as the email. This lets the graders map every commit to your ID automatically and paste marks straight into the class gradebook, so **a wrong or shared identity costs you marks**. Do not commit under a nickname or a personal Gmail.

3. Commit message tags. Start each message with the module and requirement, e.g. m2: min-max fairness model (req 2.3). One coherent slice per commit.

4. Your team seed (self-generated, nothing is handed out). Your seed is *derived deterministically from your team ID*, so you compute it yourselves on Day 1 and the instructors recompute the *identical* value at grading. Generate it once and commit it to data/seed.txt:

```
python tools/make_seed.py C02011-261-L-2252107 > data/seed.txt
```

(tools/make_seed.py is a 3-line BLAKE2 hash of your team ID, shipped in the template repo.) Use that seed for *every* random choice (simulated preferences, soft-constraint weights, any tiling), so your instance is unique yet **reproducible**: `python run_all.py --seed $(cat data/seed.txt)` must reproduce your reported numbers exactly. **Hard-coding numbers instead of reading the seed is a reproducibility failure.**

Suggested repository organization

```
C02011-261-L-2252107/                                     # your team ID = repo name
  README.md                                                # how to run, team & roles, AI-use declaration
  requirements.txt                                         # pinned versions
  run_all.py                                               # THE entry point: reproduces every reported number
  CHECKPOINTS.md                                           # weekly dated log, per-member tasks
  CONTRIBUTIONS.md                                         # who-did-what ledger + signed % table
  DECISIONS.md                                             # your own-words design log (see 4.4)
  MEETINGS.md                                              # dated weekly meeting minutes + task split
  data/                                                    # dataset, seed.txt, your seeded simulated data
  m1_logic/                                                # predicate spec, CNF, SAT scripts
  m2_ilp/                                                  # model, solver, tuning, baseline
  m3_automata/                                             # DFAs, minimization, regular to ILP, pumping
  m4_dynamics/                                             # recurrence, equilibrium/stability, sim
  app/                                                     # app.py (Streamlit) tying M1-M4 together
  report/                                                  # report + slides + link to 30-min video
```

Example commit history (the cadence and attribution we expect)

```
2026-09-08 An   m1: predicate signature + no-double-booking (req 1.1)
2026-09-11 Binh m1: CNF encoder for the toy instance (req 1.2)
2026-09-15 Chi  m1: logic-to-LP table; SAT unsat-core (req 1.3); tag m1
2026-09-24 Dung m2: sets/params + preprocessing (req 2.1, 2.6)
2026-10-02 An   m2: min-max fairness model in PuLP (req 2.3, 2.7)
2026-10-09 Binh m2: tuning + baseline comparison (req 2.8, 2.9); tag m2
2026-10-27 Chi  m3: DFA for "no>2 consecutive"; minimization (req 3.1-3.2)
Each commit references a requirement ID; each member owns a coherent slice; tags mark milestones.
Configure git user.name and user.email so authorship is attributable.
```

Commit discipline (graded). Every member authors *meaningful* commits every week; use branches and

pull requests; write descriptive messages that cite the requirement worked on. **Do not** squash the project into one late commit, pad with cosmetic commits, or commit under a single shared account. Graders read the *content* of the diffs, not the count.

Deliverables & format. A `report.pdf` (main body ≤ 20 pages, about 3–4 per module plus integration; put full proofs, large tables, and code listings in appendices), slides, and a ~ 30 -minute video in which *each* member presents their own module on camera and makes one change to the app against a small spec revealed at recording time; plus `README.md`, `requirements.txt` (pinned), `CHECKPOINTS.md`, `CONTRIBUTIONS.md`, `DECISIONS.md` (your own-words design log; see §4.3), and `MEETINGS.md` (dated minutes of your weekly team meetings: attendance, decisions, and the task split for the coming week). **Grading is always on the SHA of the relevant tag;** rewriting history after a deadline counts as a late submission.

Late-submission policy

Deadlines are the end of the stated Sunday (23:59). Grading reads the tag's timestamp, so push on time.

- **Weekly checkpoint** (week-01...week-14): a missing or late weekly tag costs **−1% of the course grade each**, capped at the Process & Ownership weight (18%). Catching up a later week does not restore a missed one, because the point is steady cadence.
- **Milestone tags** (m1–m5): a late milestone loses **10% of that milestone's points per day**, up to 3 days; after 3 days the milestone scores **0** but you should still finish it for the later modules.
- **Final submission** (submission): **−10% per day** of the *total* assignment grade, maximum 3 days, then not accepted (0).
- **One 48-hour “grace token” per team (a one-time late pass).** Each team may push *exactly one* deadline (any single milestone *or* the final) back by up to 48 hours with **no penalty and no reason required**. To use it, add one line to `CHECKPOINTS.md` *before* that deadline passes, for example `Using our grace token for m3 (new due date 10 Nov)`. It can be spent only once in the whole term, so save it for a real crunch.
- **Genuine emergencies** (illness with a medical certificate, a family emergency) are *not* charged against the grace token or this penalty scale. Contact your instructor with documentation and the case is handled individually.

How ownership is graded (both artifacts *and* authorship)

The instructors grade **both the artifacts** (report, model, code, proofs, app, slides, video) **and ownership**, meaning who actually produced what. Because cohorts are large, we use scalable, evidence-based ownership signals instead of individual oral vivas:

- **Per-member commit history** mapped to requirement IDs (auto-extracted: cadence, contribution distribution).
- **Contribution ledger** (`CONTRIBUTIONS.md`) plus a signed % table mapping every requirement to a primary owner.
- **Recorded individual walkthrough:** in the 30-minute video, each member explains *their own* part on camera and does one *live change* to the app on request (graded by the automated rubric pass, spot-checked by an instructor).
- **Anonymous intra-team peer evaluation** on the LMS, aggregated into an individual factor (§6).
- **Exam cross-check:** both the midterm and the final include questions on the assignment (see §5), answered *individually*. A member who cannot demonstrate authorship does not receive full marks even if the team artifact scores well.

Integrity & freeze. Plagiarism or code copying is a **Grade of 0**. Each team's own seed and parameters make solutions non-interchangeable; declare any AI-tool use in README.md. Enable branch protection on main (no force-push); at the deadline the instructors clone the repo and grade on the SHA of the submission tag, so any later history rewrite is treated as late and as a possible integrity matter.

Dividing the work: everyone builds every module

Team members have *different* strengths, so organizing the work to stay balanced, fair, and on schedule is itself a core engineering skill, and a genuinely hard one. Do it deliberately:

- **Every member must work on every module (M1–M4), not just own one silo.** You may lead on one module (in a team of five, the fifth leads integration or the report), but each member contributes code and understanding to all four, at least one non-trivial commit per module, because the midterm and final ask each of you *individually* about all four; a member who did only one module **will struggle in the exams**.
- **Meet weekly and record it.** Hold a short weekly meeting to split the coming week's load and review the last, and write dated minutes to MEETINGS.md (attendance, decisions, who does what). **These minutes are graded** and are used, with the commit history and peer evaluation, to justify a *fair* individual grade.
- **Balance load, rotate, review each other.** Distribute work by effort, not convenience; rotate who leads; review each other's pull requests so knowledge is shared. This is how you avoid one person *carrying* the team and protect yourself against a teammate's *free-riding*.
- **Real ownership, for real reasons.** Genuine shared ownership is what lets each of you answer the exam questions and what lets us grade each of you fairly. **This project is audited carefully; everything must be done for real, not for show:** fabricated logs, padded commits, or your name on work you cannot explain are integrity matters.

On using AI: allowed as a tool, not as a substitute

You may use AI assistants, and you must *declare* how in README.md. But this assignment is deliberately built so that “ask an AI to do everything and just read it” does not earn a good grade. The defenses are structural:

- **Your instance is unique.** Every team has its own seed, soft-constraint weights, and data slice, so a generic AI answer does not match your numbers. Results must be reproducible from data/seed.txt (§4); numbers that do not regenerate score as a reproducibility failure.
- **Process is graded, not just the artifact.** Fourteen weekly tagged commits with attributable authorship (§4) contribute to the 18% Process & Ownership block (weekly commits alone are 5%). A one-shot end-of-term dump is visible in the history and is penalized.
- **You must explain your own decisions.** DECISIONS.md records, in your own words, why you chose each objective, constraint, DFA, and dynamical direction, and what you rejected. Generic AI prose that does not fit *your* instance is easy to spot.
- **Individual verification.** In the recorded walkthrough each member makes a *live* change to the app on request, and the midterm and final ask assignment-linked questions answered *individually* (Appendix A). Understanding an AI answer is not the same as being able to modify it under time pressure.
- **Critique, do not copy.** Requirement 2.10 asks you to exhibit a plausible *wrong* model and a counter-example; module 3 asks for a real proof. These reward judgment about when an answer (including an AI's) is wrong, which is the actual skill.

Used well, AI is a fast tutor and reviewer. Used as a substitute, it fails the seed check, the process check, the decisions log, and the individual exam. The honest, higher-scoring path is to build it yourselves and use AI to check your work.

5. Relation to the Midterm & Final Examinations

Per the official CO2011 plan, exam theory is drawn from the same modules you apply here, so mastering the project prepares you for the exams (and discourages artifact-only participation):

- **Midterm (12–18 Oct):** MCQ on **Advanced Logic, Program Verification, and Optimization**, for example logical equivalence and predicate specification; LP duality, the simplex method¹⁰, and ILP modeling.
- **Final (14–28 Dec):** MCQ on **Automata, Dynamical Systems, and the Assignment Contents**, for example regular languages, DFA minimization, the pumping lemma; difference equations, equilibria and stability; *and questions on all four modules of your assignment*, answered individually.

The ownership-check questions in the Appendix are written in exactly the style used for these assignment-linked items, so **questions of this kind appear in both the midterm (GK) and the final (CK)**, not only at the end of the term. *Program verification* (weakest preconditions, loop invariants, program correctness) is examined in the midterm and is sourced from the lectures and Rosen §5.5; it is not a separate assignment deliverable.

Why the assignment matters beyond its 20%

On paper the assignment is **20%** of the course grade. In practice its true weight is much higher: the midterm and the final are drawn from the *same* four modules you build here, so a team that genuinely does the assignment is at the same time preparing for both exams, while a team that outsources it walks into the exams unprepared. Counting the exam questions it directly helps you answer, its real influence on your results is much larger than the nominal 20%. Do it for the learning, and the 20% plus the exam marks follow.

6. Assessment: Rubric and Auto-grading

Learning outcomes and constructive alignment

Each module maps to a measurable outcome, the activity that builds it, and the assessment that measures it, aligned with the official CO2011 course learning outcomes:

Module	On completion you can...	Course CLO	Assessed by
M1 Logic	formalize informal regulations as propositional/first-order logic and decide satisfiability	L.O.1.1–1.3	rubric M1 + midterm
M2 LP/ILP	build, solve, and justify an integer program for a real assignment problem, including a counter-example for a wrong model	L.O.2.1, 2.3, 2.4, 3.1	rubric M2 + midterm
M3 Automata	model temporal rules as regular languages, locate the regular / non-regular boundary, and prove non-regularity	L.O.2.2, 3.2	rubric M3 + final
M4 Dynamics	derive a discrete dynamical model, find and classify equilibria, and link long-run behavior to a design goal	L.O.2.4, 3.1, 3.2	rubric M4 + final
M5 Integration	deliver a working decision-support tool and communicate the results	L.O.3.2	rubric M5, report, video

¹⁰The simplex method solves the LP relaxation; duality gives bounds and the “shadow-price” interpretation. See [14], Ch. 5–6.

Weighting

Component	Weight
Module 1: Logic (specification, SAT, logic-to-LP)	12%
Module 2: Linear & Integer Programming (model, solver, tuning, baseline)	22%
Module 3: Automata (DFAs, minimization, regular→ILP, pumping)	16%
Module 4: Discrete Dynamical Systems (chosen direction, equilibrium & stability)	14%
Module 5: Integration & Web Application	6%
Report, Slides & Presentation Video	12%
Process & Ownership (weekly commits, all-module contribution, meeting minutes, peer evaluation)	18%
Total	100%

Each module is scored on three sub-criteria: **Correctness** (is the answer right?), **Rigor** (is the mathematics sound and justified?), and **Reproducibility** (does the code run and match the report?).

Two-stage automated grading. To grade a large cohort consistently, each repository is first run through a *deterministic autograder* that computes objective sub-scores: the solver runs and returns a feasible schedule; it beats the baseline fairness metric; the DFA accepts or rejects a hidden test set of schedule strings; the pumping-proof and required files are present; the dynamical simulation reaches the predicted equilibrium within tolerance; the Streamlit app launches. A second stage applies an **automated rubric-scoring pass**: given the report, code, and repo metrics against the sub-criteria rubric, it proposes a score and a written justification per criterion, and **an instructor reviews, adjusts, and signs off** every grade. Ownership metrics (commit cadence, contribution distribution, peer evaluation) are auto-aggregated into an **individual factor** $f \in [0.5, 1.1]$: your personal grade is the team grade scaled by your f , so free-riding is separated from genuine contribution. The factor is set from three signals with **published weights** (commit evidence 40%, peer evaluation 35%, individual defense 25%) mapped through a published band table (roughly: signal $\geq 0.9 \rightarrow f=1.0$, top decile $\rightarrow 1.1$, $< 0.4 \rightarrow 0.5$, near-zero contribution $\rightarrow 0.0$). The full grading criteria are **published in Appendix B of this brief** (and as a machine-readable `rubric.yaml` in the course template repo): every sub-criterion, its points, and the automated check behind it, so you can *self-grade* your repository before every milestone and leave no easy points on the table.

7. How to Do Well in This Assignment

Strong teams are not the ones that work hardest in December; they are the ones that follow a few habits from Week 1. Aim for these:

1. **Start Week 1 and commit weekly.** Push a tagged commit every week even when small. The `CHECKPOINTS.md` log is worth real marks and protects you if a teammate disappears.
2. **Split ownership early, rotate review.** Give each member a primary module but have a second member review every pull request, so knowledge and marks are shared and the final-exam cross-check holds no surprises.
3. **Self-grade with `rubric.yaml` before each milestone.** Run the published checks yourself; fix reproducibility gaps before we do.
4. **Make every number reproducible.** No hand-edited results: one command from a clean clone must regenerate every figure and table in the report.
5. **Reuse the real data (see §1.2).** Fit Modules 3–4 to the dataset rather than inventing numbers; graders reward evidence over invention.
6. **Prove your claims.** Back each optimum, DFA acceptance, and equilibrium with a runnable check and a short justification, which is exactly what the *Rigor* sub-criterion scores.

7. **Prepare individually for the exams.** The ownership-check questions (Appendix) appear in the midterm and final; a member who truly built a module will find them easy.

One-line test of a healthy project

At any week, a stranger should be able to `git clone` your repo, run one documented command, and reproduce your latest reported result. If that is true every week, you are on track for a top grade.

8. Glossary of Key Terms (English-Vietnamese)

Term (English)	Thuật ngữ (Tiếng Việt)	Meaning / Ý nghĩa
Invigilator / proctor	Giám thị / cán bộ coi thi	Staff supervising an exam session [1]. <i>Cán bộ giám sát một buổi thi.</i>
Constraint Satisfaction / NP-hard	Thỏa mãn ràng buộc / NP-khó	Finding an assignment meeting all constraints; NP-hard means at least as hard as every problem in NP [15]. <i>Tìm lời giải thỏa mọi ràng buộc; NP-khó nghĩa là khó ít nhất bằng mọi bài toán trong lớp NP.</i>
Integer Linear Program (ILP)	Quy hoạch tuyến tính nguyên	Optimize a linear objective over integer variables under linear constraints [14, 15]. <i>Tối ưu hàm mục tiêu tuyến tính trên biến nguyên với ràng buộc tuyến tính.</i>
Hard / soft constraint	Ràng buộc cứng / mềm	Inviolable rules versus preferences penalized in the objective [4]. <i>Luật bất buộc so với mong muốn bị phạt trong hàm mục tiêu.</i>
SAT / CNF / unsat core	Thỏa được / dạng chuẩn hội / lỗi bất khả thỏa	Boolean satisfiability; conjunctive normal form; a minimal unsatisfiable subset of clauses [10]. <i>Tính thỏa được Boolean; dạng chuẩn hội; tập con mệnh đề bất khả thỏa nhỏ nhất.</i>
Regular language / DFA / pumping lemma	Ngôn ngữ chính quy / ô-tô-mát hữu hạn đơn định / bổ đề bơm	A language recognized by a finite automaton; the pumping lemma proves non-regularity [11, 12]. <i>Ngôn ngữ được ô-tô-mát hữu hạn đoán nhận; bổ đề bơm chứng minh tính không chính quy.</i>
Regular (global) constraint	Ràng buộc chính quy	A constraint that a variable sequence be accepted by an automaton, decomposable into ILP [5, 6]. <i>Ràng buộc buộc một dãy biến phải được ô-tô-mát chấp nhận, phân rã được thành ILP.</i>
Difference equation / equilibrium / stability	Phương trình sai phân / điểm cân bằng / tính ổn định	A recurrence $x_{n+1} = f(x_n)$; a fixed point $x^* = f(x^*)$; stable if nearby states converge to it [16, 17]. <i>Hệ thức truy hồi; điểm bất động; ổn định nếu các trạng thái lân cận hội tụ về nó.</i>
Rolling horizon	Tầm cuốn chiếu	Re-solving a decision each period as the state evolves [18]. <i>Giải lại quyết định ở mỗi kỳ khi trạng thái thay đổi.</i>
Agile / sprint	Linh hoạt / chặng nước rút	Short iterative work cycles (plan, build, review), here one week each. <i>Chu kỳ làm việc lặp ngắn, ở đây mỗi tuần một chặng.</i>

9. References

Book titles are clickable and open the e-book in the course drive.

- [1] F. Hanum, M. A. Romliyah, T. Bakhtiar. “Exam invigilators assignment problem: a goal programming approach.” *Applied Mathematical Sciences*, 9(58):2871–2880, 2015. <https://doi.org/10.12988/ams.2015.52134>
- [2] H. Verplancke, D. Van Bulck, V. Limère, N. A. Schmid. “The (a)social exam invigilator assignment problem.” *Journal of Scheduling*, 29(3):185–202, 2026. <https://doi.org/10.1007/s10951-025-00868-7>

- [3] I. Hanafi et al. “A mixed integer linear programming for the exam-invigilator assignment problem: a case study at UPNM.” *JOIV: Int. J. on Informatics Visualization*, 8(2), 2024. <https://doi.org/10.62527/joiv.8.2.2196>
- [4] E. K. Burke, P. De Causmaecker, G. Vanden Berghe, H. Van Landeghem. “The state of the art of nurse rostering.” *Journal of Scheduling*, 7(6):441–499, 2004. <https://doi.org/10.1023/B:JOSH.0000046076.75950.0b>
- [5] G. Pesant. “A regular language membership constraint for finite sequences of variables.” In *Principles and Practice of Constraint Programming (CP 2004)*, LNCS 3258, pp. 482–495. Springer, 2004. https://doi.org/10.1007/978-3-540-30201-8_36
- [6] M.-C. Côté, B. Gendron, L.-M. Rousseau. “Grammar-based integer programming models for multiactivity shift scheduling.” *Management Science*, 57(1):151–163, 2011. <https://doi.org/10.1287/mnsc.1100.1264>
- [7] K. H. Rosen. *Discrete Mathematics and Its Applications*. 8th ed. McGraw-Hill, 2019. (Ch.1 logic & proofs; §5.5 program correctness; Ch.13 modeling computation / automata. A Student’s Solutions Manual is available.)
- [8] M. Huth, M. Ryan. *Logic in Computer Science: Modelling and Reasoning about Systems*. 2nd ed. Cambridge University Press, 2004.
- [9] M. Ben-Ari. *Mathematical Logic for Computer Science*. 3rd ed. Springer, 2012.
- [10] A. Biere, M. Heule, H. van Maaren, T. Walsh (eds.). *Handbook of Satisfiability*. 2nd ed. IOS Press, 2021.
- [11] P. Linz. *An Introduction to Formal Languages and Automata*. 6th ed. Jones & Bartlett, 2017.
- [12] J. E. Hopcroft, R. Motwani, J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Pearson, 2006.
- [13] S. Kandar. *Introduction to Automata Theory, Formal Languages and Computation*. Pearson Education India, 2016. (Includes MCQs for self-testing.)
- [14] J. Matoušek, B. Gärtner. *Understanding and Using Linear Programming*. Springer, 2007.
- [15] G. L. Nemhauser, L. A. Wolsey. *Integer and Combinatorial Optimization*. Wiley, 1999.
- [16] F. R. Giordano, W. P. Fox, S. B. Horton. *A First Course in Mathematical Modeling*. 5th ed. Cengage, 2013.
- [17] S. Elaydi. *An Introduction to Difference Equations*. 3rd ed. Springer, 2005.
- [18] S. Sethi, G. Sorger. “A theory of rolling horizon decision making.” *Annals of Operations Research*, 29:387–415, 1991. <https://doi.org/10.1007/BF02283607>
- [19] Software. Google OR-Tools: <https://developers.google.com/optimization>. PuLP: <https://coin-or.github.io/pulp/>. SCIP: <https://scipopt.org>. Streamlit: <https://docs.streamlit.io>.

Good luck. Plan weekly, own your part, and enjoy building something real. Post questions on the LMS forum.

A. Appendix: Sample Ownership-Check Questions

*These are representative of the individual, assignment-linked questions. Questions of this kind are asked in **both the midterm (GK) and the final (CK)**, not only at the end of the term. They are deliberately hard and specific to work you actually do: a member who genuinely built the module will find them approachable, whereas a free-rider will not. **Answers are intentionally withheld**; use them to prepare and to test each other within the team.*

Module 1: Logic

- Q1.** Encoding “shift s needs exactly two of n invigilators” in CNF with a brute-force (binomial) at-most-two part: how many clauses does the at-most-two part contribute for that one shift, and why does this become impractical at the real data scale?
- (A) $\binom{n}{3}$ (B) $\binom{n}{2}$ (C) n (D) $2n$
- Q2.** Your SAT check on a data slice is UNSAT with a minimal unsatisfiable core = {one availability, one capacity, one eligibility clause}. Which single conclusion is justified?
- (A) The whole dataset is infeasible.
(B) The ILP is unbounded.
(C) *That slice* is infeasible, and feasibility requires dropping or softening *one* of the three rules; no objective or LP relaxation can help.
(D) The LP relaxation is feasible iff the constraint matrix is totally unimodular.

Module 2: Linear & Integer Programming

- Q1.** On {3 staff members, 4 unit-demand shifts, at most 2 shifts each}, compare the min-max objective with the L_1 -deviation objective (around the mean $\bar{w} = 4/3$). Which is correct?
- (A) Min-max has a *unique* optimum (2, 1, 1).
(B) Both objectives are optimized only by (2, 1, 1).
(C) Min-max is optimized by *several* profiles with peak 2 (for example (2, 1, 1) and (2, 2, 0)), while the L_1 objective selects (2, 1, 1) uniquely.
(D) L_1 is optimized by (2, 2, 0).
- Q2.** Let Z_{LP} be the LP-relaxation optimum of your (minimization) ILP and Z_{ILP} the integer optimum. Which is always true?
- (A) $Z_{ILP} \leq Z_{LP}$.
(B) $Z_{LP} \leq Z_{ILP}$, and a zero gap at the root node means no branching is needed.
(C) $Z_{LP} = Z_{ILP}$ always.
(D) The gap is negative when all variables are binary.

Module 3: Automata & Formal Languages

- Q1.** Over $\Sigma = \{W, -\}$, how many states does the minimal *complete* DFA (with a trap state) that accepts exactly the schedules with *no more than two consecutive working days* have?
- (A) 3 (B) 4 (C) 5 (D) Infinitely many
- Q2.** Exactly one of these rules is *not* regular. Which one, and what intersection language proves it via the pumping lemma?
- (A) “At most 3 shifts in any rolling 7-day window”, via $\{a^n b^n\}$.
(B) “Equal numbers of morning and afternoon shifts over the whole term”, via $L \cap M^* A^* = \{M^n A^n\}$.
(C) “Never two night shifts in a row”, via $\Sigma^* N N \Sigma^*$.
(D) None is non-regular.

Module 4: Discrete Dynamical Systems

Q1. For $F_{n+1} = 0.75 F_n + 2 \cdot \text{load} - 0.5 \cdot \text{rest}$ with a steady load of 4 and rest of 1 per week, the equilibrium fatigue and its stability are:

- (A) $F^* = 30$, stable (since $|0.75| < 1$). (B) $F^* = 40$, stable.
 (C) $F^* = 30$, unstable. (D) No equilibrium exists.

Q2. In the rolling-horizon model $w_{n+1} = w_n + A_n x_n$, why does w_n have no non-trivial fixed point, and what converges under a fair weekly policy?

- (A) Its Jacobian is I (eigenvalue 1), so w_n grows without bound while the *normalized* workload spread tends to 0.
 (B) Eigenvalue 0, so $w_n \rightarrow 0$.
 (C) w_n converges to the mean load.
 (D) Nothing converges.

(An instructor answer key is maintained separately and is not part of the student release.)

B. Appendix B: Grading Rubric (self-grade before every milestone)

This is the human-readable form of the machine-readable `rubric.yaml` shipped with the brief.

Checked by: *auto* = deterministic autograder, *ai* = automated rubric-scoring proposal reviewed and signed by an instructor, *human* = instructor. Personal grade = team score $\times$ individual factor $f \in [0.5, 1.1]$ (0.0 for confirmed free-riding). Reproducibility from a clean clone is a **gate** on all code points.

Component (weight)	Pts	Checked by	What is checked
Module 1: Logic (12)			
Predicate specification	4	ai	Hard rules as correct first-order formulas.
SAT / unsat core	4	auto	Model on SAT slice; minimal core on UNSAT.
Logic-to-LP bridge	4	ai	Clauses mapped to linear 0/1 (in)equalities.
Module 2: Linear & Integer Programming (22)			
Model correctness	6	ai	Sets, variables, objective, constraints justified.
Solver feasibility	6	auto	Feasible schedule honoring all hard constraints.
Beats baseline	6	auto	Fairness metric strictly better than baseline.
Counter-example	4	ai	A wrong model + concrete invalid schedule.
Module 3: Automata (16)			
DFAs vs. hidden tests	5	auto	≥ 3 DFAs accept/reject the hidden test set.
Product & minimization	4	auto	Reported state counts match recomputation.
Regular constraint to ILP	4	ai	Correct decomposition; re-solved.
Pumping-lemma proof	3	ai	Non-regular rule proven, right language.
Module 4: Discrete Dynamical Systems (14)			
Recurrence (fit to data)	5	ai	Direction justified; recurrence from real data.
Equilibrium	5	auto	Simulation reaches predicted fixed point.
Stability	4	ai	Stability classified; linked to fairness.

Component (weight)	Pts	Checked by	What is checked
Module 5: Integration & Web App (6)			
App launches	3	auto	<code>streamlit run app/app.py</code> loads a dataset.
Integration quality	3	ai	M1 check, M2 plan, M3 flag, M4 trajectory in one page.
Report, Slides & Video (12)			
Report	6	ai	Clear; numbers match code; ≤ 20 pages main body.
Video	6	human	Each member presents; one live app change.
Process & Ownership (18)			
Weekly commits	5	auto	Tagged commit + CHECKPOINTS.md each of 14 weeks.
All-module contribution	4	auto	Each member: ≥ 1 non-trivial authored commit in <i>each</i> of the four modules.
Meeting minutes	5	human	Dated MEETINGS.md (attendance, decisions, weekly task split), consistent with commits.
Ledger & peer eval	4	human	Signed %-table consistent with history + peers.
Total	100		Gated by reproducibility.

Integrity: plagiarism or code copying = grade 0; a shared or fake git identity loses attribution points; rewriting history after a deadline is treated as late plus an integrity review.